

Product Performance Analysis Using MySQL

OBJECTIVE : To analyze product-level performance using SQL techniques such as window functions and CTEs, focusing on ranking, growth trends, and performance variability across products.

Tools Used:

MySQL — Data analysis, SQL queries, CTEs, and window functions

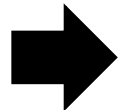
Microsoft Power BI — Data visualization and dashboard creation

Dataset Overview

The datasets used in this project were sourced from **Kaggle** and are publicly available for educational and analytical purposes

Calendar Dataset Total Records: 731 Columns: • Date • Year • Month • Day • Week • Day of Week Purpose: • Time-based analysis • Seasonal trend tracking	Products Dataset Total Records : 200 Columns: • Product ID • Product Name • Category • Brand • Price Purpose: • Product performance analysis • Category-wise comparison	Customers Dataset Total Records : 5000 Columns: • Customer ID • Age • Gender • Loyalty Member • Join Date Purpose: • Customer segmentation • Loyalty analysis	Sales Dataset Total Records : 1000000 Columns: • Sales ID • Product ID • Customer ID • Store ID • Quantity Sold • Revenue • Sales Date Purpose: • Revenue analysis • Sales trend analysis	Stores Dataset Total Records : 100 Columns: • Store ID • Store Name • Location • Region • Store Type Purpose: • Store-wise performance analysis • Regional comparison
--	---	---	---	---

Data Model



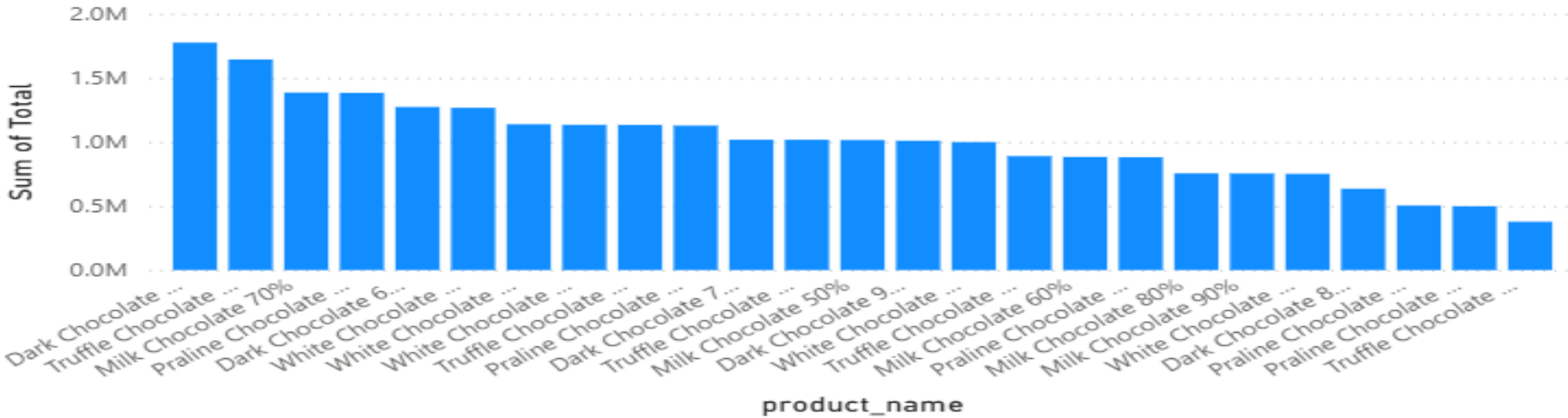
Key Relationships:

- Customers → customer_id
- Products → product_id
- Stores → store_id
- Calendar → date
- Sales acts as the central fact table

Q1. Calculate total revenue generated by each product.

```
select p.product_name,  
       round(sum(s.revenue),2) as Total  
from sales as s  
inner join products as p  
on s.product_id = p.product_id  
group by p.product_name  
order by sum(s.revenue) desc ;
```

Sum of Total by product_name

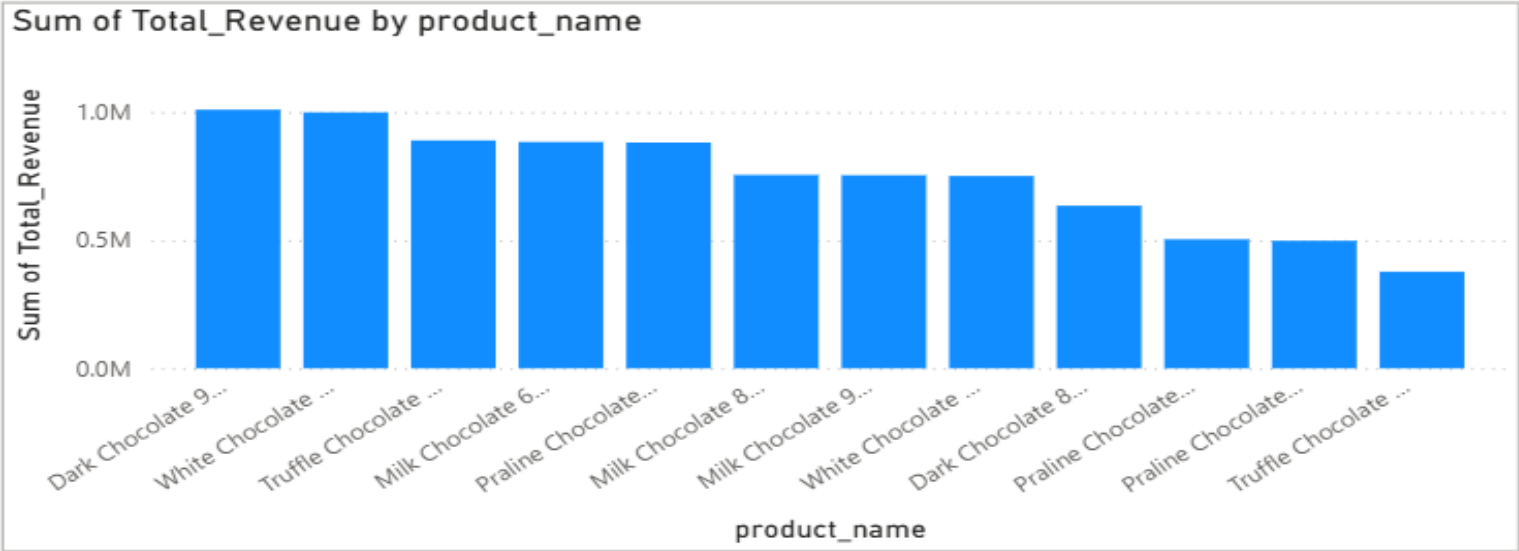


product_name	Total
Dark Chocolate 50%	1775041.34
Truffle Chocolate 80%	1643258.83
Milk Chocolate 70%	1384758.27
Praline Chocolate 50%	1382597.04
Dark Chocolate 60%	1272649.97
White Chocolate 50%	1265949.75
White Chocolate 90%	1137968.57
White Chocolate 80%	1133285.39
Truffle Chocolate 90%	1132883.8
Praline Chocolate 70%	1127404.94
Dark Chocolate 70%	1017229.64
Truffle Chocolate 50%	1017102.43
Milk Chocolate 50%	1014751.12
Dark Chocolate 90%	1009248.87
White Chocolate 70%	998795.58
Truffle Chocolate 70%	889254.2
Milk Chocolate 60%	883264.52
Praline Chocolate 90%	881139.19
Milk Chocolate 80%	754675.84
Milk Chocolate 90%	753459.18
White Chocolate 60%	750658.69
Dark Chocolate 80%	634638.76
Praline Chocolate 80%	503618.28
Praline Chocolate 60%	497887.59
Truffle Chocolate 60%	377126.43

Q2. Identify products whose total revenue is below the overall average revenue across all products.

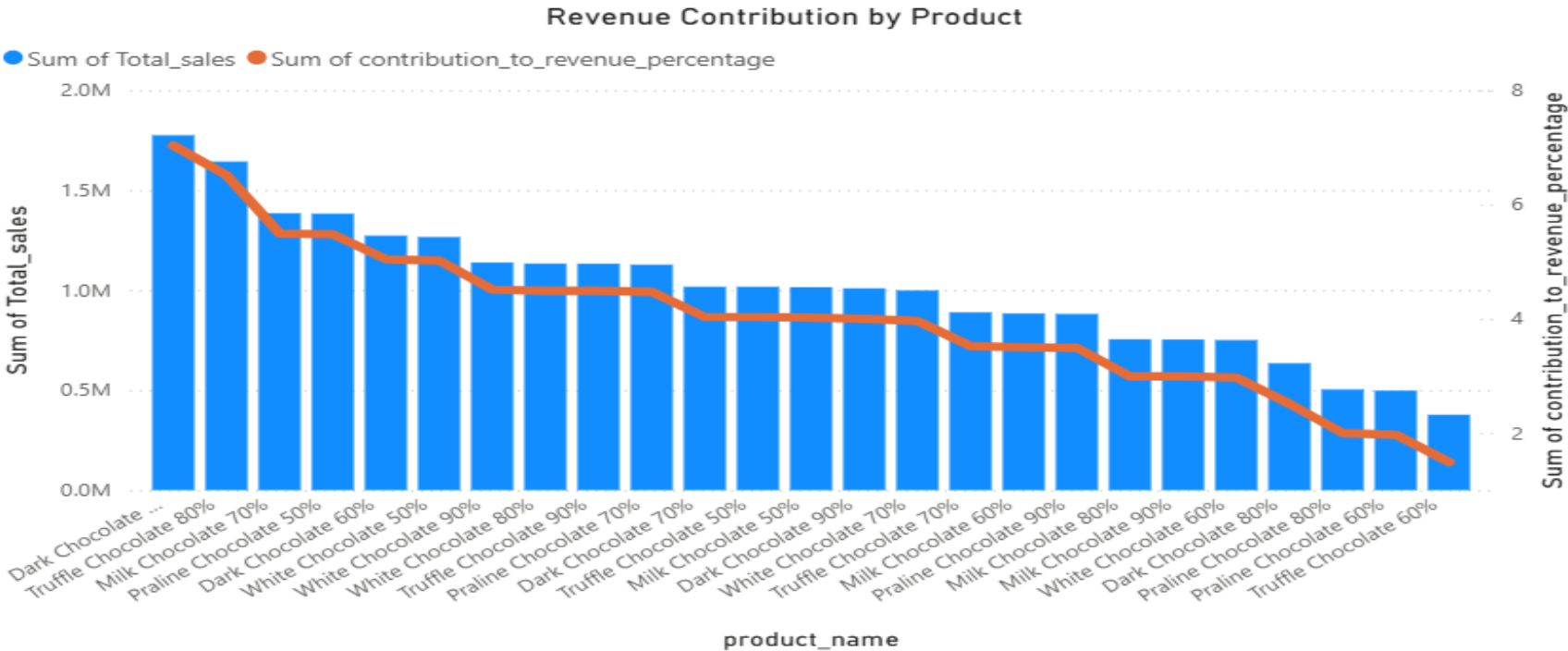
```
with cte as (  
  select p.product_name,  
         sum(s.revenue) as Total  
    from sales as s  
   inner join products as p  
     on s.product_id = p.product_id  
   group by p.product_name)  
select product_name,  
       round(Total,2) as Total_Revenue  
  from cte  
 where Total < (select avg(Total) from cte) ;
```

product_name	Total_Revenue
Milk Chocolate 90%	753459.18
Praline Chocolate 90%	881139.19
Praline Chocolate 80%	503618.28
Truffle Chocolate 70%	889254.2
Dark Chocolate 90%	1009248.87
Truffle Chocolate 60%	377126.43
White Chocolate 70%	998795.58
Milk Chocolate 60%	883264.52
Dark Chocolate 80%	634638.76
Praline Chocolate 60%	497887.59
Milk Chocolate 80%	754675.84
White Chocolate 60%	750658.69



Q3 : Calculate the percentage contribution of each product to total revenue.

```
select p.product_name,
       round(sum(s.revenue),2) as Total_sales,
       round(sum(s.revenue)*100/sum(sum(s.revenue))over(),2)
       as contribution_to_revenue_percentage
from sales as s
     inner join products as p
     on s.product_id = p.product_id
group by p.product_name
order by total_sales desc ;
```

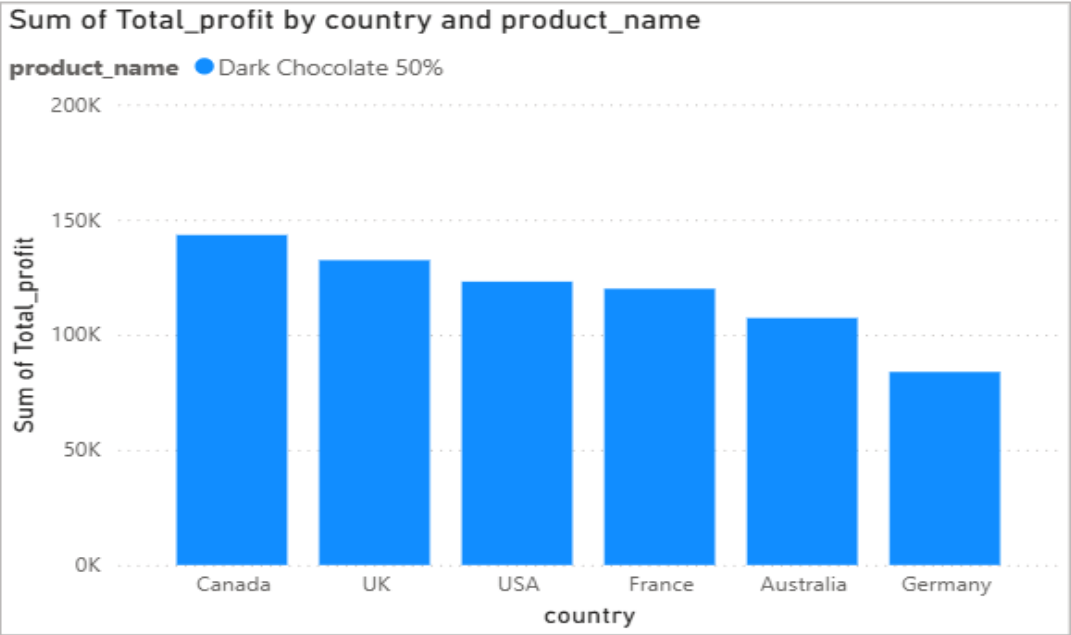


product_name	Total_sales	contribution_to_revenue_percentage
Dark Chocolate 50%	1775041.34	7.03
Truffle Chocolate 80%	1643258.83	6.51
Milk Chocolate 70%	1384758.27	5.49
Praline Chocolate 50%	1382597.04	5.48
Dark Chocolate 60%	1272649.97	5.04
White Chocolate 50%	1265949.75	5.02
White Chocolate 90%	1137968.57	4.51
White Chocolate 80%	1133285.39	4.49
Truffle Chocolate 90%	1132883.8	4.49
Praline Chocolate 70%	1127404.94	4.47
Dark Chocolate 70%	1017229.64	4.03
Truffle Chocolate 50%	1017102.43	4.03
Milk Chocolate 50%	1014751.12	4.02
Dark Chocolate 90%	1009248.87	4
White Chocolate 70%	998795.58	3.96
Truffle Chocolate 70%	889254.2	3.52
Milk Chocolate 60%	883264.52	3.5
Praline Chocolate 90%	881139.19	3.49
Milk Chocolate 80%	754675.84	2.99
Milk Chocolate 90%	753459.18	2.99
White Chocolate 60%	750658.69	2.97
Dark Chocolate 80%	634638.76	2.51
Praline Chocolate 80%	503618.28	2
Praline Chocolate 60%	497887.59	1.97
Truffle Chocolate 60%	377126.43	1.49

Q4 Which product earns the highest profit in each country?

```
select * from (  
    select st.country, p.product_name,  
        round(sum(s.profit),2) as Total_profit,  
        dense_rank() over (partition by st.country order by sum(s.profit) desc) as rnk  
        from stores as st  
        inner join sales as s  
            on st.store_id = s.store_id  
        inner join products as p  
            on p.product_id = s.product_id  
    group by st.country, p.product_name ) as high  
where rnk = 1;
```

country	product_name	Total_profit	rnk
Australia	Dark Chocolate 50%	107234.28	1
Canada	Dark Chocolate 50%	143310.58	1
France	Dark Chocolate 50%	119957.82	1
Germany	Dark Chocolate 50%	83822.81	1
UK	Dark Chocolate 50%	132372.72	1
USA	Dark Chocolate 50%	123044.46	1



Q5 : Find the top 3 products within each category based on total revenue.

```
with product_sales as
(
    select p.category, p.product_name,
        round(sum(s.revenue),2) as total_revenue
    from sales as s
        inner join products as p
            on s.product_id = p.product_id
    group by p.category, p.product_name ),

rank_table as (
    select category, product_name, total_revenue,
        dense_rank() over (partition by category order by total_revenue desc) as Rnk
    from product_sales )

select Category, Product_name,
    Total_revenue,Rnk
    from rank_table
        where Rnk <= 3 ;
```

Category	Product_name	Total_revenue	Rnk
Dark	White Chocolate 90%	510464.7	1
Dark	Truffle Chocolate 80%	504925.63	2
Dark	Praline Chocolate 70%	504855.67	3
Milk	White Chocolate 50%	381163	1
Milk	Truffle Chocolate 50%	379976.73	2
Milk	White Chocolate 90%	377414.92	3
Praline	Milk Chocolate 70%	878013.4	1
Praline	Dark Chocolate 50%	633849.41	2
Praline	Truffle Chocolate 80%	509426.23	3
Truffle	Praline Chocolate 50%	377489.81	1
Truffle	Dark Chocolate 60%	256033.4	2
Truffle	Truffle Chocolate 80%	255902.32	3
White	Truffle Chocolate 90%	508497.32	1
White	Dark Chocolate 60%	507296.32	2
White	Dark Chocolate 70%	383249.68	3

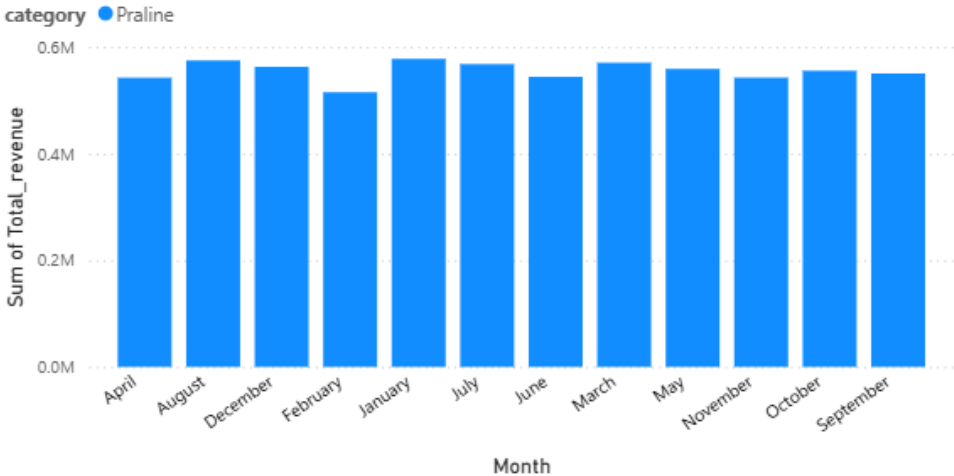


Q6 : Which product categories peak in which months?

```
with group_table as (  
    select monthname(c.`date`) as Months,  
           p.category,  
           round(sum(s.revenue),2) as Total_revenue  
    from calendar as c  
    inner join sales as s  
        on c.`date` = s.order_date  
    inner join products as p  
        on p.product_id = s.product_id  
    group by monthname(c.`date`) , p.category )  
  
select * from (  
    select months as Month, category,  
           Total_revenue,  
           dense_rank() over (partition by months order by Total_revenue desc) as rnk  
    from group_table ) as rank_table  
where rnk = 1;
```

Month	category	Total_revenue	rnk
January	Praline	578041.14	1
August	Praline	574965.41	1
March	Praline	570760.63	1
July	Praline	568205.4	1
December	Praline	563022.61	1
May	Praline	559204.6	1
October	Praline	555769.13	1
September	Praline	550662.94	1
June	Praline	544303.51	1
November	Praline	542786.42	1
April	Praline	542612.46	1
February	Praline	515307.07	1

Sum of Total_revenue by Month and category

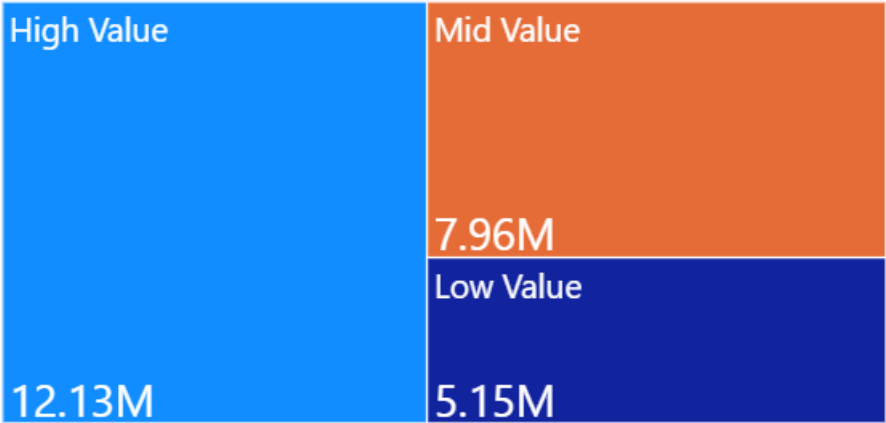


Q7 : Categorize products by revenue and analyze their quantity

```
with revenue as (  
    select p.product_name,  
           round(sum(s.revenue),2) as Total_sales,  
           sum(s.quantity) as Total_quantity  
    from sales as s  
      inner join products as p  
        on s.product_id = p.product_id  
   group by p.product_name ),  
  
Tiered_data as (  
    select *,  
           ntile(3) over (order by Total_sales desc) Tile  
    from revenue )  
  
select Product_name,  
       Total_sales, Total_quantity,  
       case  
         when tile = 1 then "High Value"  
         when tile = 2 then "Mid Value"  
         else "Low Value"  
       end Performance_tier  
from Tiered_data  
 order by total_sales desc;
```

Product_name	Total_sales	Total_quantity	Performance_tier
Dark Chocolate 50%	1775041.34	208996	High Value
Truffle Chocolate 80%	1643258.83	193431	High Value
Milk Chocolate 70%	1384758.27	162335	High Value
Praline Chocolate 50%	1382597.04	162670	High Value
Dark Chocolate 60%	1272649.97	149346	High Value
White Chocolate 50%	1265949.75	149170	High Value
White Chocolate 90%	1137968.57	134217	High Value
White Chocolate 80%	1133285.39	133171	High Value
Truffle Chocolate 90%	1132883.8	133692	High Value
Praline Chocolate 70%	1127404.94	132564	Mid Value
Dark Chocolate 70%	1017229.64	119825	Mid Value
Truffle Chocolate 50%	1017102.43	120033	Mid Value
Milk Chocolate 50%	1014751.12	119141	Mid Value
Dark Chocolate 90%	1009248.87	118969	Mid Value
White Chocolate 70%	998795.58	117771	Mid Value
Truffle Chocolate 70%	889254.2	104445	Mid Value
Milk Chocolate 60%	883264.52	103564	Mid Value
Praline Chocolate 90%	881139.19	103964	Low Value
Milk Chocolate 80%	754675.84	88784	Low Value
Milk Chocolate 90%	753459.18	88507	Low Value
White Chocolate 60%	750658.69	88704	Low Value
Dark Chocolate 80%	634638.76	74734	Low Value
Praline Chocolate 80%	503618.28	59422	Low Value
Praline Chocolate 60%	497887.59	58771	Low Value
Truffle Chocolate 60%	377126.43	44244	Low Value

Sum of Total_sales by Performance_tier



Challenges Faced

- **Handling 1 Million Sales Records:** Processing large datasets significantly increases query execution time and causes performance bottlenecks during heavy analytical tasks.
- **Managing Complex Table Joins:** Linking multiple relational tables requires precise syntax to prevent duplicate records and ensure data integrity across dimensions.
- **Writing Advanced SQL Queries:** Constructing deep analytical structures using window functions and Common Table Expressions (CTEs) increases the risk of logical errors.
- **Performing Time-Based Analysis:** Tracking chronological data patterns requires complex sequential logic to accurately evaluate growth trends across specific months.

Recommendations

- **Track Growth Trends:** Compare monthly sales patterns to see if top-performing items like **Dark Chocolate 50%** maintain steady revenue momentum or experience sharp monthly drops.
- **Measure Performance Variability:** Compute revenue variance across the 12 months to quantify which items have stable demand versus fluctuating items like **Praline**.
- **Expand Ranking Insights:** Cross-reference total quantity sold against total sales revenue to spotlight high-volume, lower-margin items like **Milk Chocolate 60%**.
- **Evaluate Tier Transitions:** Analyze monthly data to see if low-performing products like **Truffle Chocolate 60%** shift into higher value tiers during your highest-revenue peak month of January.